

競賽小抄

用法：先用資料範圍排除不可能的複雜度，再從題目特徵找技巧，最後檢查模板與邊界。本頁只濃縮本站已完成的技巧筆記，不取代各篇的證明與完整說明。

思考與實作

競賽策略

開始比賽後先瀏覽全部題目並粗分難度，再依成功機率分配時間；不要按照題號一路卡到底。

階段	行動	時間提醒
掃題	瀏覽所有題目，標記簡單 / 中等 / 困難	比賽開始立即進行
讀題	確認限制、部分分與初步方向	中等以下約 10 分鐘；難題約 15 分鐘
簡單題	想法清楚就優先完成	約 15 分鐘內
中等題	有可行方向才繼續推進	約 20 分鐘內
困難題	確定值得投入再深入	約 30 分鐘內
部分分	能快速實作就先取得	約 10 分鐘內

時間不是硬性規定，而是「重新評估」的鬧鐘。時間到時問自己：

- 想法是否已能完整說明並證明？
- 剩下的是單純實作，還是其實仍在猜作法？
- 是否有更簡單或更高成功率的題目？
- 能否先取得部分分？

若沒有明確進展，記錄目前結論後跳題，稍後再回來。

從想法到實作

不要想到大概方向就立刻開始寫。依序完成：

1. **用範例驗證**：逐步模擬自己的演算法，確認得到範例答案。
2. **主動找反例**：嘗試用最小資料、極端資料、相同值與特殊結構推翻想法。
3. **簡化模型**：刪掉不必要的狀態與分支，清楚說出資料結構、狀態和轉移。
4. **寫下程式架構**：模型仍複雜時，先列出函式、迴圈順序、維護量與邊界。
5. **開始實作**：每完成一部分就確認不變量仍成立。

開始寫程式前，至少能回答：

- 每個變數 / DP 狀態的精確意義是什麼？
- 資料處理順序為什麼能滿足依賴關係？
- 哪些操作決定時間與空間複雜度？
- 最可能錯的是公式、邊界、初始化，還是資料型別？

卡住時快速分類

現象	可能原因	下一步
範例就不對	狀態或公式有誤	手算每一步，找第一個分歧點
範例對但無法證明	可能只是猜到規律	主動縮小資料並找反例

現象	可能原因	下一步
作法正確但程式很難寫	模型或狀態過度複雜	重新命名狀態、拆函式、畫出轉移順序
一直抓不到 bug	不知道哪個不變量壞掉	分段輸出中間結果，建立最小失敗測資
同一題停留太久	沉沒成本影響判斷	對照時間上限，記錄進度後換題

時間複雜度辨認

從資料範圍估算

以下以 C++、一般 1~2 秒時限作粗略判斷；常數、語言、記憶體與測資形狀都會影響結果。

最大資料量	優先考慮	常見技巧
$n \leq 20$	$O(2^n)$ 、 $O(n2^n)$	子集合枚舉、狀態壓縮
$n \leq 500$	$O(n^3)$	2D1D 區間 DP、Floyd-Warshall
$n \leq 5 \times 10^3$	$O(n^2)$	1D1D / 2D0D DP、枚舉 pair
$n \leq 2 \times 10^5$	$O(n \log n)$	排序、二分搜、BIT、線段樹、Dijkstra、Kruskal
$n \leq 10^6$	$O(n)$ 、 $O(n \log n)$ (注意常數)	前綴和、差分、雙指標、單調隊列
n 很大但答案範圍可搜	$O(T \log V)$	二分答案； T 是檢查一次的成本

常見複雜度速查

技巧	時間複雜度	備註
一維前綴和	建表 $O(n)$ ，詢問 $O(1)$	靜態區間和
差分	修改 $O(1)$ ，還原 $O(n)$	離線區間修改
雙指標 / 滑動窗口	排序後 $O(n)$	每個指標只能單向移動
單調隊列	$O(n)$	每個元素至多進出一次
二分搜	$O(\log V)$ 次檢查	必須先證明單調性
0/1 背包	$O(nW)$	W 是容量 / 總和
區間 DP	2D0D : $O(n^2)$; 2D1D : $O(n^3)$	是否額外枚舉切點
倍增	預處理 $O(n \log K)$ ，查詢 $O(\log K)$	把步數拆成二進位
BIT	修改、前綴詢問皆 $O(\log n)$	適合可逆的前綴資訊
線段樹	修改、區間詢問皆 $O(\log n)$	節點資訊須可合併
Dijkstra	$O((V + E) \log V)$	不可有負權邊
Bellman-Ford	$O(VE)$	可處理負權並偵測負環
Floyd-Warshall	$O(V^3)$	所有點對最短路
Kruskal	$O(E \log E)$	排序邊後以 DSU 合併
質因數分解	單次 $O(\sqrt{N})$	少量、大數字
SPF 質因數分解	預處理後單次 $O(\log N)$	大量、小範圍詢問

題型辨認

序列與區間

題目特徵	優先想到	快速判斷
多次查靜態區間和 / XOR	前綴和	$[l, r]$ 由兩個前綴相減 / XOR

題目特徵	優先想到	快速判斷
多次離線區間加值	差分	改 <code>diff[1]</code> 與 <code>diff[r+1]</code> ，最後還原
連續區間和恰為 k 的數量	前綴和 + 等價類計數	找先前出現過多少個 <code>pre - k</code>
矩形和恰為 k	固定上下界 + 壓成一維	再做連續區間和計數
已排序序列的配對、計數或極值	雙指標	一端變大時，另一端移動方向是否固定？
最長 / 最短合法連續區間	滑動窗口	加右端、刪左端能否維護合法性？
固定窗口或距離限制內極值	單調隊列	隊首過期；隊尾維持單調性
排序後找第一個 $\geq x$ / 第一個 $> x$	<code>lower_bound</code> / <code>upper_bound</code>	關聯容器要用成員函式
單點更新 + 區間詢問	BIT / 線段樹	前綴和選 BIT；一般可合併資訊選線段樹
區間更新 + 單點詢問	差分觀念 / 線段樹	是否需要線上回答？
相鄰交換、逆序對、抽取順序	Swap Linked Technique	把交換次數轉成逆序對或抽取成本

二分搜與最佳化

題目特徵	技巧	檢查事項
最大化最小值 / 最小化最大值	二分答案	<code>check(x)</code> 是否呈 FFFFTTTT 或 TTTTFFFF
求第 k 小 / 大	二分答案	計算 $\leq x$ 或 $\geq x$ 的個數
單峰 / 單谷函數極值	三分搜	只能用在 unimodal；整數最後暴力檢查小範圍
看似要枚舉所有 pair	數學式化簡	把只含 i 、只含 j 的項分開，嘗試降成 $O(n)$
最小化絕對值總和 / 曼哈頓距離	中位數	偶數個點時最優解可能是一段區間

動態規劃

題目特徵	狀態方向	注意
容量 / 總和限制下選物品	背包 DP	0/1 倒序；無限背包正序
答案由連續區間子問題合併	<code>dp[1][r]</code> 區間 DP	依區間長度由小到大
切成數段，每段有成本	分段 DP	枚舉完成段或最後一段；非法初值設極值
有報酬的互斥區間	加權區間排程 DP	依右端排序，二分最後一個不衝突區間
樹上選點、斷邊、連通塊	Tree DP	子樹是否獨立？不獨立則做子樹合併
數字範圍 $[0, N]$ 且限制與各位數有關	Digit DP	常見狀態：位置、限制上界、前綴零、題目狀態
總分固定的雙人零和賽局	定和賽局 / Negamax	一方最大化等價另一方最小化
重複走 K 步相同轉移	倍增	預處理 2^j 步後到哪裡

設計 DP 時先寫一句完整定義：`dp[狀態]` := 已處理範圍與答案意義。把狀態視為點、轉移視為有向邊時，依賴關係必須是 DAG。

圖與樹

題目特徵	優先想到	注意
有先後依賴、判斷有向環	拓撲排序	處理完的點數少於 V 即有環
無向圖找環	DFS + 父節點	遇到已訪且不是父邊的點
有向圖找環	DFS 三色 / 遞迴堆疊	走到仍在堆疊中的點是 back edge
單源、非負權最短路	Dijkstra	pop 後跳過過期距離；負權不可用
單源、有負權	Bellman-Ford	第 V 輪仍可鬆弛代表負環

題目特徵	優先想到	注意
所有點對最短路、點數小	Floyd-Warshall	枚舉中繼點必須在最外層
連接所有點的最小總成本	MST	Kruskal：邊排序 + DSU；不連通則無生成樹
最小化路徑上的最大邊	MBST / MST 性質	也可能是最小瓶頸路徑詢問
樹上兩點路徑、距離	LCA	$dist(u, v) = dep[u] + dep[v] - 2dep[lca]$
子樹總和 / 極值 + 區間資料結構	Euler Tour	子樹壓成 <code>[tin, tout)</code>
邊依時間加入 / 消失	圖論建模	可把時間視為邊權，再接最短路、MST、LCA
使用某資源才可通過特定邊	分層圖	將資源狀態加入節點

數學與幾何

題目特徵	技巧 / 公式
多集合聯集計數	排容：加單集合、減兩兩交集、交替正負
恰好為 k 的計數	用「至少 / 至多」的答案做差分
依同餘、差值、前綴性質分類計數	為每個等價類設 key，以 map 計數
大量組合數	巴斯卡三角形或模數下的組合數預處理
模數除法	不能直接除；需乘模反元素且確認存在
線性轉移做很多次	矩陣快速幂
期望總和	期望值線性性： $E[X + Y] = E[X] + E[Y]$ ，不要求獨立
向量夾角 / 投影	內積
左右轉 / 有向面積	外積正負號
曼哈頓距離限制	座標轉換後嘗試切比雪夫距離
六邊形網格移動 / 旋轉	Axial / Cube 座標，避免直接在畫面座標硬判

模板寫法

基本骨架

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

}
```

本機除錯可加 `-Wall -Wextra -Wshadow -D_GLIBCXX_DEBUG`，但送出前記得移除 `_GLIBCXX_DEBUG`。

二分答案 (找第一個 true)

```
int low, hi;

while (low <= hi)
{
```

```

int mid = (low + hi) / 2;
if (check(mid)) low = mid + 1;
else hi = mid - 1;
}

```

- 先寫清楚 `check` 的真假定義與答案落在哪一側。
- 上界、下界必須覆蓋答案；加法可能溢位時使用 `long long`。

前綴和與差分

```

// 0-based, pre[i] = a[0..i) 的總和
vector<long long> pre(n + 1);
for (int i = 0; i < n; ++i) pre[i + 1] = pre[i] + a[i];
long long range_sum = pre[r + 1] - pre[l]; // [l, r]

// [l, r] 全部加 v
diff[l] += v;
if (r + 1 < n) diff[r + 1] -= v;

```

滑動窗口

```

for (int l = 0, r = 0; r < n; ++r) {
    add(a[r]);
    while (!valid()) remove(a[l++]);
    // 此時 [l, r] 合法；依題意更新答案
}

```

只有「右端加入後，持續移動左端就能恢復合法」時才能套用。

0/1 背包

```

vector<long long> dp(W + 1, NEG);
dp[0] = 0;
for (auto [w, value] : items)
    for (int j = W; j >= w; --j)
        dp[j] = max(dp[j], dp[j - w] + value);

```

- 0/1 背包容量倒序，避免同一物品重複使用。
- 若狀態表示「恰好裝到」，不可達狀態不能初始化成 0。

Dijkstra

```

using P = pair<long long, int>;
priority_queue<P, vector<P>, greater<P>> pq;
vector<long long> dis(n, INF);
dis[s] = 0;
pq.push({0, s});
while (!pq.empty()) {

```

```

auto [d, u] = pq.top(); pq.pop();
if (d != dis[u]) continue;
for (auto [v, w] : g[u]) {
    if (dis[v] > d + w) {
        dis[v] = d + w;
        pq.push({dis[v], v});
    }
}
}
}

```

Floyd-Warshall

```

for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (dis[i][k] != INF && dis[k][j] != INF)
                dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);

```

k 必須在最外層；加法前避免 $\text{INF} + \text{INF}$ 溢位。

BIT

```

struct BIT {
    int n;
    vector<long long> bit;
    BIT(int n) : n(n), bit(n + 1) {}
    void add(int x, long long v) {
        for (++x; x <= n; x += x & -x) bit[x] += v;
    }
    long long prefix(int r) { // [0, r)
        long long ans = 0;
        for (; r > 0; r -= r & -r) ans += bit[r];
        return ans;
    }
    long long query(int l, int r) { // [l, r)
        return prefix(r) - prefix(l);
    }
};

```

DSU

```

struct DSU {
    vector<int> p, sz;

    DSU(int n) : p(n), sz(n, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int leader(int x) {

```

```

    return p[x] == x ? x : p[x] = leader(p[x]);
}
bool same(int x, int y) {
    return leader(x) == leader(y);
}
int size(int x) {
    return sz[leader(x)];
}
bool merge(int x, int y) {
    x = leader(x), y = leader(y);
    if (x == y) return false;
    if (sz[x] < sz[y]) swap(x, y);
    p[y] = x;
    sz[x] += sz[y];
    return true;
}
};

```

merge 回傳是否真的合併 · Kruskal 可直接用它判斷是否選邊。

線段樹

```

template<class T, T (*op)(T, T), T (*e)()>
struct SegTree {
    int n;
    vector<T> node;

    SegTree(int size) : n(1) {
        while (n < size) n <= 1;
        node.assign(2 * n, e());
    }
    void set(int p, T x) {
        p += n;
        node[p] = x;
        while (p >= 1) node[p] = op(node[p << 1], node[p << 1 | 1]);
    }
    T get(int l, int r) { // [l, r)
        T left = e(), right = e();
        for (l += n, r += n; l < r; l >= 1, r >= 1) {
            if (l & 1) left = op(left, node[l++]);
            if (r & 1) right = op(node[--r], right);
        }
        return op(left, right);
    }
};

long long op(long long a, long long b) { return a + b; }
long long e() { return 0LL; }

```

```
// SegTree<Long Long, op, e> seg(n);
```

線段樹節點設計檢查 寫模板前先決定三件事：

1. Node 精確保存什麼資訊？
2. merge(left, right) 是否滿足結合律？
3. 空區間的單位元素是什麼？例如 sum 是 0、min 是 INF。

統一使用 0-based 與半開區間 $[l, r)$ ，能減少邊界錯誤。

Sparse Table (靜態 RMQ)

```
template<class T>
struct SparseTable {
    int n, lg;
    vector<int> log2;
    vector<vector<T>> st;

    SparseTable(const vector<T>& a) : n((int)a.size()) {
        log2.assign(n + 1, 0);
        for (int i = 2; i <= n; ++i) log2[i] = log2[i / 2] + 1;
        lg = log2[n] + 1;
        st.assign(lg, vector<T>(n));
        st[0] = a;
        for (int k = 1; k < lg; ++k)
            for (int i = 0; i + (1 << k) <= n; ++i)
                st[k][i] = min(st[k - 1][i],
                                st[k - 1][i + (1 << (k - 1))]);
    }

    T query(int l, int r) { // min of [l, r) · 要求 l < r
        int k = log2[r - l];
        return min(st[k][l], st[k][r - (1 << k)]);
    }
};
```

重疊兩段查詢只適合 min/max/gcd 這類冪等運算；一般結合運算需拆成互斥區間。

BFS 與 DFS

```
// BFS：入隊時標記，避免同一點重複入隊
vector<int> dis(n, -1);
queue<int> q;
dis[s] = 0;
q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : g[u]) if (dis[v] == -1) {
        dis[v] = dis[u] + 1;
        q.push(v);
    }
}
```

```

    }
}

// 無向樹 DFS
auto dfs = [&](auto&& self, int u, int parent) -> void {
    for (int v : g[u]) {
        if (v == parent) continue;
        self(self, v, u);
    }
};
dfs(dfs, root, -1);

```

一般圖 DFS 要用 `visited`; 只有確認輸入是樹時才可只排除父節點。

Euler Tour (子樹壓平)

```

vector<int> tin(n), tout(n), order;
auto dfs = [&](auto&& self, int u, int parent) -> void {
    tin[u] = (int)order.size();
    order.push_back(u);
    for (int v : g[u]) {
        if (v == parent) continue;
        self(self, v, u);
    }
    tout[u] = (int)order.size();
};
dfs(dfs, root, -1);
// u 的子樹對應 order 的 [tin[u], tout[u])

```

有向圖找環 (三色 DFS)

```

vector<int> color(n); // 0= 未進入 · 1= 遞迴路徑中 · 2= 已離開
auto dfs = [&](auto&& self, int u) -> bool {
    color[u] = 1;
    for (int v : g[u]) {
        if (color[v] == 1) return true;
        if (color[v] == 0 && self(self, v)) return true;
    }
    color[u] = 2;
    return false;
};

bool has_cycle = false;
for (int i = 0; i < n; ++i)
    if (color[i] == 0 && dfs(dfs, i)) has_cycle = true;

```

樹直徑 (無權樹)

```
auto farthest = [&](int s) {
    vector<int> dis(n, -1);
    queue<int> q;
    dis[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (dis[v] == -1) {
            dis[v] = dis[u] + 1;
            q.push(v);
        }
    }
    int p = max_element(dis.begin(), dis.end()) - dis.begin();
    return pair{p, dis[p]};
};

auto [u, ignored] = farthest(0);
auto [v, diameter] = farthest(u);
```

輸入必須是一棵連通樹；加權樹則把 BFS 改成累加邊權的 DFS。

全排列、子集合與折半枚舉

```
sort(a.begin(), a.end());
do {
    // 使用目前排列
} while (next_permutation(a.begin(), a.end()));

for (int mask = 0; mask < (1 << n); ++mask) {
    for (int i = 0; i < n; ++i) {
        if (mask >> i & 1) {
            // i 在此子集合中
        }
    }
}
```

- 全排列為 $O(n! \cdot n)$ ；從排序後的排列開始，才能枚舉全部排列。
- 位元遮罩的 $1 \ll n$ 使用 `int`，通常只適合很小的 n ；較大時改用 `1LL << n`，仍要注意位數上限。
- $O(2^n)$ 太慢但 n 約為 40 時，可把集合切兩半，分別枚舉後排序 / 二分。

質數篩與質因數分解

```
vector<bool> is_prime(N + 1, true);
is_prime[0] = is_prime[1] = false; // 要求  $N \geq 1$ 
for (long long i = 2; i * i <= N; ++i)
    if (is_prime[i])
        for (long long j = i * i; j <= N; j += i)
```

```

        is_prime[j] = false;

vector<pair<long long, int>> factorize(long long x) {
    vector<pair<long long, int>> factors;
    for (long long p = 2; p <= x / p; ++p) {
        if (x % p) continue;
        int count = 0;
        while (x % p == 0) x /= p, ++count;
        factors.push_back({p, count});
    }
    if (x > 1) factors.push_back({x, 1});
    return factors;
}

```

使用 $p \leq x / p$ 可避免 $p * p$ 溢位。

區間內倍數與平方數個數

對正整數 d 與非負區間 $[l, r]$:

```

long long multiples = r / d - (l - 1) / d;

long long floor_sqrt(long long x) {
    long long r = sqrtl((long double)x);
    while ((r + 1) <= x / (r + 1)) ++r;
    while (r > x / r) --r;
    return r;
}
long long squares = floor_sqrt(r) - (l == 0 ? 0 : floor_sqrt(l - 1));

```

若區間允許負數 · C++ 整數除法是朝零截斷 · 倍數公式必須另寫 floor division。

分數

```

struct Fraction {
    long long num, den;

    Fraction(long long num = 0, long long den = 1) : num(num), den(den) {
        assert(den != 0);
        if (den < 0) this->num = -this->num, this->den = -this->den;
        long long g = gcd(abs(this->num), this->den);
        this->num /= g;
        this->den /= g;
    }
    friend bool operator<(const Fraction& a, const Fraction& b) {
        return (__int128)a.num * b.den < (__int128)b.num * a.den;
    }
    friend Fraction operator+(const Fraction& a, const Fraction& b) {
        return {a.num * b.den + b.num * a.den, a.den * b.den};
    }
}

```

```

    }
    friend Fraction operator-(const Fraction& a, const Fraction& b) {
        return {a.num * b.den - b.num * a.den, a.den * b.den};
    }
    friend Fraction operator*(const Fraction& a, const Fraction& b) {
        return {a.num * b.num, a.den * b.den};
    }
    friend Fraction operator/(const Fraction& a, const Fraction& b) {
        assert(b.num != 0);
        return {a.num * b.den, a.den * b.num};
    }
};

```

比較已用 `__int128` 防交叉相乘溢位；四則運算本身仍可能溢位，數值範圍大時要先約分或改用大整數。

二維向量

```

struct Vec {
    long long x, y;
    Vec operator+(const Vec& b) const { return {x + b.x, y + b.y}; }
    Vec operator-(const Vec& b) const { return {x - b.x, y - b.y}; }
};

long long dot(Vec a, Vec b) {
    return a.x * b.x + a.y * b.y;
}

long long cross(Vec a, Vec b) {
    return a.x * b.y - a.y * b.x;
}

long double length(Vec a) {
    return hypot((long double)a.x, (long double)a.y);
}

long double point_segment_distance(Vec p, Vec a, Vec b) {
    Vec ab = b - a;
    if (dot(ab, p - a) <= 0) return length(p - a);
    if (dot(a - b, p - b) <= 0) return length(p - b);
    return abs((long double)cross(ab, p - a)) / length(ab);
}

```

若座標乘積可能超過 `long long`，`dot`、`cross` 改用 `__int128`；線段退化成一時需另外處理。

矩陣

```

struct Matrix {
    int n, m;
    vector<vector<ll>> mat;
    int MOD = 1e9 + 7;

    // 加法單位元素

```

```

Matrix (int _n, int _m, bool multiply = 0) : mat(_n, vector<ll>(_m, -INF)) {
    n = _n; m = _m;

    if (multiply) {
        assert(n == m);
        *this = identity(n);
    }
}

// 乘法單位元素
Matrix identity (int sz) {
    Matrix res(sz, sz);
    for (int i = 0; i < sz; i++) res.mat[i][i] = 1;
    return res;
}

Matrix operator*(const Matrix &b) const {
    assert(m == b.n);

    Matrix res(n, b.m);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < b.m; j++) {
            for (int k = 0; k < m; k++) {
                res.mat[i][j] += mat[i][k] * b.mat[k][j];
                if (res.mat[i][j] >= MOD) res.mat[i][j] %= MOD;
            }
        }
    }

    return res;
}

// 方陣快速冪
Matrix pow(ll t) {
    assert(n == m);

    Matrix res = identity(n);
    Matrix base = *this;

    while (t) {
        if (t & 1) res = res * base;
        base = base * base;
        t >>= 1;
    }

    return res;
}

```

```
void print() {
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) cout << this->mat[i][j] << " \n"[j]
}
};
```

上場前 30 秒檢查

- 空集合、單一元素、全部相同、嚴格遞增 / 遞減是否正確？
- 圖可能不連通？可能有重邊、自環、負權或無法到達？
- `int` 是否會溢位？乘法前是否已轉成 `long long`？
- DP 不可達狀態是否用 `INF` / `-INF`？轉移前是否判斷可達？
- 排序是否破壞原編號？相等元素的順序是否重要？
- 使用的是 `[1,r]` 還是 `[1,r)`？`lower_bound` / `upper_bound` 有沒有混用？
- 遞迴 DFS 深度是否可能爆 `stack`？

尚未收錄 / 待完成

以下技巧檔目前沒有可濃縮的完整內容，因此未放進小抄：

- [DAG DP](#)：檔案目前是空的。
- [DSU](#)：檔案目前是空的；MST 筆記雖有使用 DSU，但缺少獨立整理。
- [進位制](#)：目前只有 TODO 連結。
- [mindset_and_implementation/index.md](#)：目前只有 TODO 連結。
- [implementation_rule.md](#)：「找反例」說明、常見例子與代表題目尚未完成。